

1. Given `a = np.arange(1, 26).reshape(5, 5)`, replace the middle 3×3 block with -1.
2. Given `a = np.arange(1, 37).reshape(6, 6)`, extract every alternate row and every alternate column, starting from index 0.
3. Given `a = np.array([12, 5, 18, 7, 25, 3, 14, 9, 30])`, replace values < 10 with -1 and values > 20 with 1.
4. Given `a = np.array([[12, 45, 23, 17], [56, 21, 34, 10], [18, 39, 27, 44]])`, find the maximum value of each row and its corresponding index.
5. Given `a = np.array([[2, 4, 6, 8], [1, 3, 5, 7], [10, 20, 30, 40]])`, find the minimum of each column and the index of each minimum.
6. Given `a = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]])`, exchange the first and last rows using NumPy indexing.
7. Using the same array, exchange the first and last columns.
8. Given `a = np.arange(1, 17).reshape(4, 4)`, extract the four corner elements as a 1-D array.
9. Given `a = np.arange(1, 26).reshape(5, 5)`, extract both diagonals.
10. Using the same array, replace the main diagonal with 0 without using a loop.
11. Given `a = np.array([4, 7, 2, 4, 9, 7, 3, 2, 8, 7])`, find the unique elements that occur more than once.
12. Given `a = np.array([15, 8, 22, 3, 17, 30, 11, 6, 25])`, extract elements that are even and greater than 10.
13. Given `a = np.arange(1, 13).reshape(3, 4)`, find the sum of each row and each column.
14. Using the same array, find the mean of each row and subtract the corresponding row mean from every element.
15. Given `a = np.array([[34, 12, 56, 23], [78, 45, 11, 67], [29, 90, 54, 18]])`, sort every row in ascending order.
16. Using the same array, sort every column in ascending order.
17. Given `a = np.array([[10, 20, 30], [40, 50, 60]])`, append the row [70,80,90] using NumPy.
18. Using the same array, append the column [70,80].
19. Given `a = np.arange(1, 13)`, reshape it into 3×4 and then into 2×6 .
20. Given `a = np.arange(1, 17).reshape(4, 4)`, create a new array containing only elements greater than the mean of `a`.
21. Given `a = np.array([[4, 8, 12], [16, 20, 24], [28, 32, 36]])`, find the row having the largest sum.
22. Using the same array, find the column having the smallest sum.
23. Given `a = np.array([[2, 5, 8], [1, 4, 7], [3, 6, 9]])`, find the elements that are greater than their row mean.
24. Given `a = np.array([[10, 15, 20], [25, 30, 35], [40, 45, 50]])`, replace every element divisible by 5 with 0.
25. Given `a = np.array([[8, 1, 6], [3, 5, 7], [4, 9, 2]])`, check whether this is a magic square using NumPy operations only. No for loop.
26. Given `a = np.array([[8, 1, 6], [3, 5, 7], [4, 9, 3]])`, modify the previous magic-square checker so that it correctly reports this as not a magic square.

27. Given `a = np.arange(1, 26).reshape(5, 5)`, create a 1-D array containing all row sums, all column sums, the main diagonal sum, and the opposite diagonal sum. Then determine whether all these values are equal.

28. Given `a = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]])`, create `[10 50 90]` using NumPy indexing. Then create `[30 50 70]` without manually writing the values.

29. Given `a = np.array([[4, 9, 2], [3, 5, 7], [8, 1, 6]])`, determine whether every row, every column, and both diagonals have the same sum. Use one combined NumPy expression as far as reasonably possible.

30. Given `a = np.array([[10, 20, 30, 40], [50, 60, 70, 80], [90, 100, 110, 120], [130, 140, 150, 160]])`, without a loop produce `[10 40 70 100 130 160]` using NumPy indexing/slicing. Then produce `[40 70 100 130]`.